CLAUDE CODE SKILL · SCROLL-3D-HERO

SCROLL 3D HERO

A product that floats in the hero, revolves as you scroll and lands in a product card. This guide covers how kanaknaturals.com builds its plate hero, the dot-matrix footer from unitedcarriers.com, animated characters that walk through a page, and the reusable skill that puts all of it on any website.



The skill's template page at progress 0. The fibre plate is generated in code (no model download), lit like a product shot and drifting with a slow idle wobble.

01 The effect in one minute
02 How kanaknaturals.com does it
03 The demo, frame by frame
04 Installing and using the skill
05 Markup and CSS
06 Choreographing keyframes
07 Options, API and CSS hook
08 Recipes, models and frameworks
09 Pitfalls and checks
10 What's in the skill folder
11 Dot-matrix footer wordmark
12 Animated characters
13 Dot story

REPOSITORY	RUNTIME	DATE
faisalqadir/faisalqadir	Three.js r170, vanilla ES module	October 2026

01 THE EFFECT IN ONE MINUTE

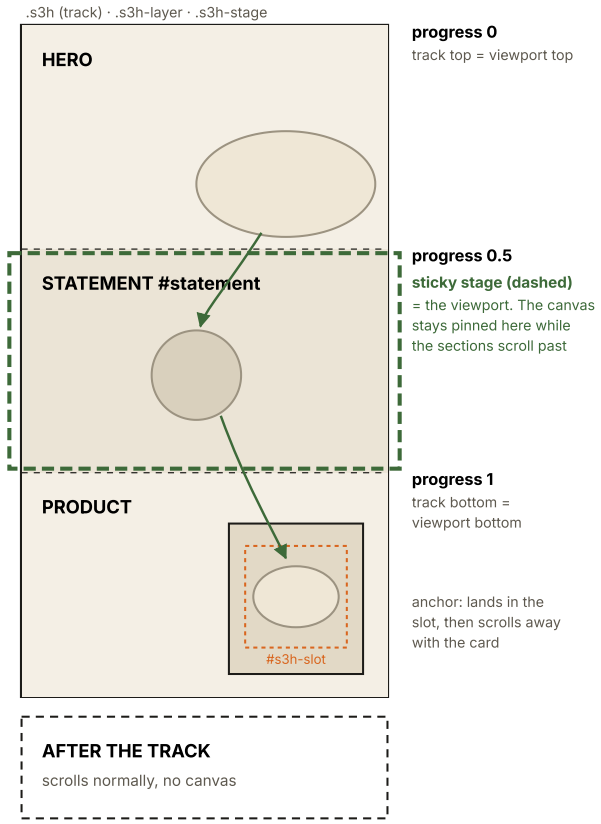
A WebGL canvas is pinned over the first few sections of the page. As the visitor scrolls through those sections, scroll progress moves a 3D object along a set of keyframes.

FOUR MOVING PARTS

1. **Track.** A wrapper around every section the object travels through. Its scroll range is progress `0 → 1`.
2. **Sticky stage.** An absolutely positioned layer spans the track. Inside it a `position: sticky`, `100vh` stage holds the canvas, so the canvas stays put while the sections scroll underneath. `pointer-events: none` keeps the links underneath clickable.
3. **Keyframes.** Progress drives position, size, rotation and shadow through eased keyframes, with light damping so mouse-wheel steps glide.
4. **Anchor.** The final keyframe follows a DOM element, so the object lands exactly in the product card at every screen size.

EXTRAS BUILT IN

- Idle wobble and float, so the object is never frozen.
- Optional turntable spin and pointer tilt.
- `setModel()` swaps products with a revolving transition.
- A `--s3h-progress` CSS variable for syncing DOM animations.
- Built-in procedural plate and bowl, plus GLB loading with Draco or Meshopt compression.
- Respects reduced-motion settings, pauses when off-screen, and shows a fallback when WebGL is missing.



The track is three screens tall, so it has two screens of scroll range. The dashed stage stays in the viewport the whole time; only the content underneath moves.

02 HOW KANAKNATURALS.COM DOES IT

Findings from reading the site's production code: the homepage HTML, its CSS and the JavaScript chunk behind the hero (`HeroThree`) with its dependencies.

Method. The code the site already sends to every browser (its HTML, CSS and JavaScript files) was downloaded and read, following the static JavaScript workflow of the *reverse-engineer-anything* (REA) skill, which is installed in this session. REA's own command-line analyser was not run, because this cloud session's permission policy blocks running downloaded third-party packages. All the numbers below come straight from the minified source.

STACK

- Astro pages with React islands; **React Three Fiber** renders the scene.
- drei `Environment` lit by a warehouse HDR image, and `ContactShadows`.
- framer-motion `scroll()` measures scroll progress; Lenis provides smooth scrolling.
- Models are Draco-compressed GLBs (`gltfjsx --transform`), one per product.

LAYOUT

- `.home-hero-three`: absolutely positioned over the hero, the "Your reputation is our pride" statement and the product cards; `z-index: 99`, `pointer-events: none`.
- `.home-hero-three-stick`: `position: sticky; top: 0; height: 100vh`.
- Camera: field of view 30°, positioned at z = 5.

BEHAVIOUR

- **Hero:** the plate is huge, tilted towards the viewer and partly off-screen at the bottom right, with a slow wobble (an eased `cos(t/2)` of about 3.6°).
- **Scrolling down:** it flips upside down, shrinks to about a third of its size and moves to the centre.
- **Arriving:** the whole canvas slides sideways to the product card, the plate finishes almost a full revolution and grows slightly into the card.
- **In the card:** switching products spins the models out and in around the vertical axis while scaling them through zero.
- A decorative orange fork floats in the background on its own path.

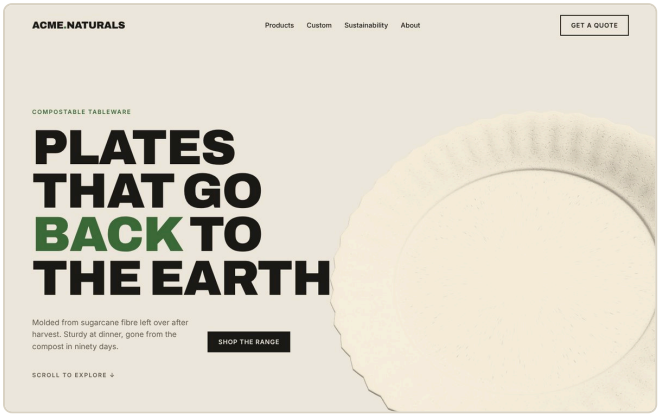
THEIR NUMBERS, TRANSLATED TO THIS SKILL'S UNITS

PHASE	ON KANAKNATURALS.COM	AS A SCROLL-3D-HERO KEYFRAME
Start	position (1.6, −0.65), rotation π·(0.25, −0.28, 0.165), scale 16	`at: 0, x: 0.67, y: -0.49, rotate: [45, -50, 30]`
0 → 40%	move to (−0.3, 0); rotation → π·(−1, 0, −0.05)	`at: 0.4, x: -0.13, y: 0, rotate: [-180, 0, -9]`
0 → 60%	scale 16 → 5	`at: 0.6, size` ≈ 31% of the start size
40 → 100%	canvas slides to the card centre; rotation → π·(−1.91, 0.33, 0)	`at: 1, anchor: '#slot', rotate: [-344, 59, 0]`
60 → 100%	scale 5 → 7	`fit` sizes it to the card
Idle	rotation x/y eased towards cos(t/2)·0.02π	`idle: { wobble: 3.6, speed: 0.5 }`

The skill follows the same overall structure and choreography, then adds three improvements. Positions are relative to the viewport, so one set of keyframes works at every size. The card landing uses an anchor instead of hand-measured pixel offsets. Breakpoints are handled with media-query overrides instead of dividing every number by a per-device factor. No models, images or code from the site are included; the plate is generated in code.

03 THE DEMO, FRAME BY FRAME

Screenshots from `scripts/preview.mjs` on the template page, desktop at 1440 × 900. Each frame was taken after the plate had come to rest.



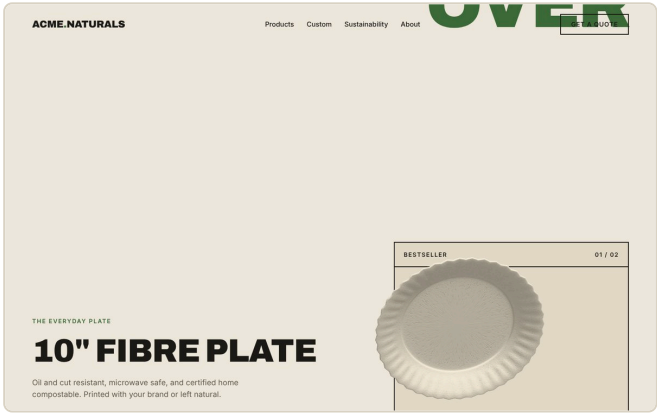
Progress 0 · hero. Large, tilted 60° towards the viewer, bleeding off the bottom right, clear of the headline and button.



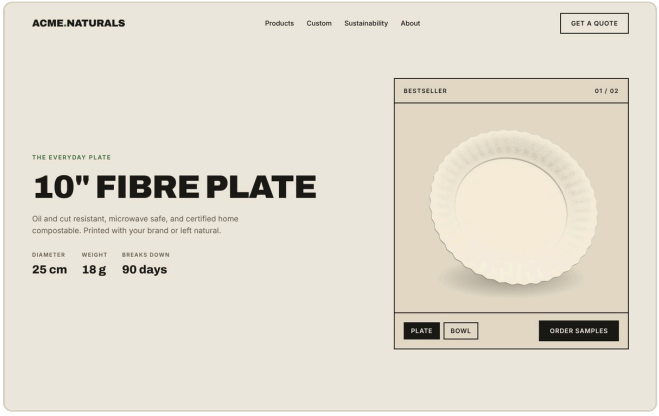
0.3 · flipping. Rotating backwards, the underside now faces the camera. The hero copy fades out through `--s3h-progress`.



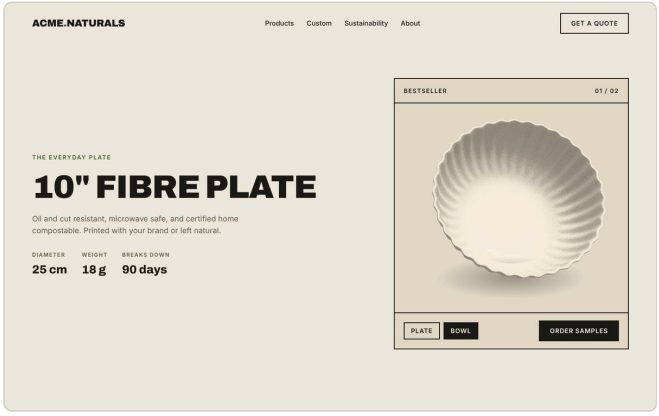
0.5 · statement. The keyframe `at: '#statement'` fires as the section reaches the top: the plate is smaller, centred and upside down.



0.8 · travelling. Heading for the card slot, which it tracks live as the product section scrolls in, still revolving.

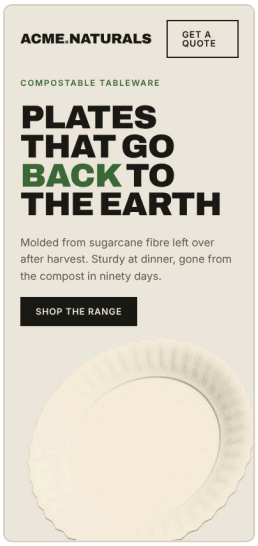


Progress 1 · landed. Upright again after one full revolution (−295° = 65°), sized to the slot with `fit: 0.92`, with the soft shadow faded in.



Product switch. Clicking "Bowl" calls `hero.setModel({ shape: 'bowl' })`: the plate revolves out and the procedural bowl revolves in.

MOBILE, 390 × 844, WITH ITS OWN KEYFRAMES



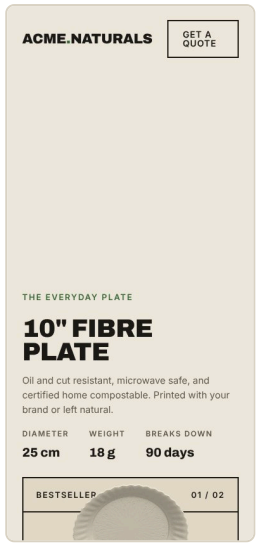
0 · below the headline



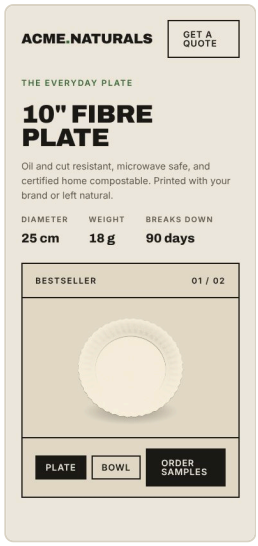
0.3 · flipping



0.5 · under the statement



0.8 · travelling



1 · in the card

The same page uses a `responsive` override for `(max-width: 767px)`. The plate starts below the headline, stays under the statement copy instead of covering it, and lands in the same anchored slot. Pointer tilt is switched off in the mobile override.

Also verified: a Draco-compressed glTF loaded from a CDN, auto-centred and scaled up from a few centimetres to fill the card; zero console errors at every stop on desktop and mobile; settled-state detection for reliable screenshots.

04 INSTALLING AND USING THE SKILL

INSTALL

- **In this repository:** it's already at `.claude/skills/scroll-3d-hero/`, so any Claude Code session opened in the repo picks it up.
- **For every project:** copy the folder to `~/.claude/skills/`, or install the packaged `scroll-3d-hero.skill` file through claude.ai's "Save skill" button.

INVOKE

Just describe what you want; the skill triggers on 3D, WebGL, spinning or scroll-animation requests. For example:

- "Make the plate in my hero flip as you scroll and land in the product card, like kanaknaturals.com."
- "Next.js site: spin `/models/cup.glb` in the hero and zig-zag it beside the feature sections."
- Or type `/scroll-3d-hero`.

WHAT CLAUDE THEN DOES

1. **Brief:** pins down the object (your GLB, or the procedural plate/bowl), the path through your sections, and your stack.
2. **Runtime and markup:** copies `scroll-3d-hero.js` in and wraps your sections in the track and stage.
3. **Choreography:** writes keyframes, landing on an anchor in your card.
4. **Phones:** adds `responsive` keyframes.
5. **Verification:** runs the preview script, looks at every frame, and iterates until nothing covers the copy and the landing is clean.

05 MARKUP AND CSS

```
<div class="s3h">                <!— track —>
  <div class="s3h-layer" aria-hidden="true">
    <div class="s3h-stage"></div>  <!— canvas
here —>
  </div>
  <section class="hero">…</section>
  <section id="story">…</section>
  <section id="product">
    … <div id="s3h-slot"></div> …
  </section>
</div>
```

```
.s3h { position: relative; }
.s3h-layer { position: absolute; inset: 0;
  z-index: 3; pointer-events: none; }
.s3h-stage { position: sticky; top: 0;
  height: 100vh; overflow: hidden; }
.s3h-canvas { display: block; width: 100%;
  height: 100%; opacity: 0;
  transition: opacity .9s ease; }
.s3h-stage.is-ready .s3h-canvas { opacity: 1; }
```

In plain HTML, add an import map for `three` and `three/addons/` (pinned to a version, as in the template), then `import { createScroll3DHero } from './scroll-3d-hero.js'`. With a bundler, run `npm i three` and skip the import map. To put the object *behind* the text, give the layer `z-index: 0` and the sections `position: relative; z-index: 1`.

06 CHOREOGRAPHING KEYFRAMES

```
keyframes: [
  { at: 0, x: 0.58, y: -0.3, size: 0.95, rotate: [60, -30, 20], shadow: 0 },   // hero: big, tilted
  { at: '#statement', x: -0.08, y: 0.04, rotate: [-110, 20, -12] },          // flipped over
  { at: 0.62, size: 0.42 },                                                  // shrink on its own schedule
  { at: 0.85, shadow: 0 },
  { at: 1, anchor: '#s3h-slot', fit: 0.92, rotate: [-295, 0, 0], shadow: 1 },  // lands after a full turn
]
```

KEY	MEANING
`at`	Progress 0–1, or a selector: "when this element's top reaches the top of the viewport".
`x` , `y`	Centre of the object: −1 is the left/bottom edge, 0 the centre, 1 the right/top edge.
`size`	Largest dimension as a fraction of the viewport's shorter side. Models are auto-normalised, so this means the same for any model.
`rotate`	[x, y, z] in degrees, interpolated per axis, so −360 is a full revolution. `rx` / `ry` / `rz` set one axis.
`z`	Depth in world units, towards the camera at z = 5. For zoom-through moves.
`shadow`	0–1 opacity of the soft shadow. Fade it in only where the object rests on something.
`anchor` , `fit`	Follow an element's centre every frame (x/y become offsets); `fit` sizes the object to that fraction of the element's shorter side.
`ease`	`easeInOut` (default), `easeIn` , `easeOut` , `linear` or a function. Applies to the segment ending at this keyframe.

Two rules that do most of the work. First, each property interpolates only among the keyframes that set it, so staggering size, rotation and shadow makes motion feel expensive. Second, for flat objects, `rotate: [0,0,0]` is edge-on and `[90,0,0]` faces the camera; pass through edge-on angles mid-segment and land around 55–70°.

07 OPTIONS, API AND CSS HOOK

```
const hero = createScroll3DHero({
  track: '.s3h', stage: '.s3h-stage',
  // model: { shape: 'plate' | 'bowl', color, ribs, ribDepth, roughness, texture }
  //    or { url: 'product.glb', rotate: [x, y, z], prepare(obj) {} }  or { object }
  model: { shape: 'plate' },
  keyframes: [...],
  responsive: { '(max-width: 767px)': { keyframes: [...], tilt: 0 } },
  camera: { fov: 30, distance: 5 },
  idle: { wobble: 3.5, speed: 0.5, float: 0.015 }, // or false
  spin: 0,                                         // deg/s turntable spin
  tilt: 0,                                         // deg lean towards the pointer
  damping: 7,                                      // 0 = locked to the scrollbar (use with Lenis)
  shadow: { opacity: 0.35, width: 0.9, offset: 0.42, color: '#1b1a17' },
  lighting: { environment: 0.6, key: 1.8, exposure: 1, toneMapping: 'neutral' },
  maxPixelRatio: 2, dracoPath: null, debug: false, // or ?s3h-debug in the URL
  onReady(hero) {}, onLoadProgress(f) {}, onUpdate(progress, pose) {},
});
```

RETURNED API

- `hero.progress`, `hero.settled`, `hero.model`
- `hero.setModel(source, { direction })`: revolving product swap; rapid calls are queued, and stale ones dropped.
- `hero.setKeyframes(list)`, `hero.refresh()`, `hero.destroy()`
- `hero.ready` (a promise), plus `scene`, `camera` and `renderer` for extra objects.
- The stage gets `is-ready`, `data-s3h-ready` and `data-s3h-settled`, and fires the event `s3h:ready`.

SYNC DOM WITH CSS ONLY

The track carries `--s3h-progress` (0–1), updated every frame:

```
.hero-copy { opacity:
  calc(1 - var(--s3h-progress, 0) * 3); }
.card-cta { opacity: clamp(0,
  calc((var(--s3h-progress, 0) - .82) * 6), 1); }
```

08 RECIPES, MODELS AND FRAMEWORKS

RECIPES (REFERENCES/CHOREOGRAPHY.MD)

- **Land in a card:** the template's path, from hero through story to card.
- **Turn-around reveal:** spin a bottle or box on `ry` to show its back label.
- **Zig-zag:** alternate `x: ±0.5` at each feature section, opposite the copy.
- **Turntable:** `spin: 25` with a near-static pose.
- **Zoom-through:** animate `z` to 4.2, then hide.
- **Product switcher:** previous/next buttons calling `setModel` with a direction.

MODELS (REFERENCES/MODELS.MD)

- Sources: the client's own CAD files, Blender (most packaging is a spun profile), phone scans, AI image-to-3D tools, licensed marketplaces. Never the inspiring site's files.
- Export GLB, +Y up, transforms applied, textures at 2k or less.
- Compress to under 2–3 MB:

```
npx @gltf-transform/cli optimize in.glb out.glb \
  --compress draco --texture-compress webp \
  --texture-size 2048
```

FRAMEWORKS (REFERENCES/FRAMEWORKS.MD)

Plain HTML, Vite, React/Next.js (`useEffect` + dynamic import + `destroy`), Vue/Nuxt, Svelte, Astro, WordPress, Webflow and Shopify.

09 PITFALLS AND CHECKS

SYMPTOM	CAUSE AND FIX
Object scrolls away instead of pinning	`overflow: hidden` / `auto` on an ancestor (often `body` or a page builder's wrapper) disables sticky. Use `overflow-x: clip`. The runtime prints a console warning naming the element.
Nothing moves	The track is no taller than the viewport, so there is no scroll range. Plan two or three screens of sections inside it.
Lands off-screen	Progress 1 is reached when the track's bottom meets the viewport's bottom. Keep the slot in the last viewport-height: a final section of about `min-height: 100vh` with the card centred.
Floaty, double-smoothed motion	A smooth-scroll library is already easing the scroll. Set `damping: 0`.
Blank page locally	ES modules don't load from `file://`. Serve over http (the preview script does this).
Slow first paint	The model is too heavy. Compress to under 2–3 MB with Draco/Meshopt and WebP; KTX2 textures aren't supported.
Covers the headline	Move the progress-0 keyframe away from the copy, let the object bleed off the edge, and use `responsive` on phones.

Accessibility. The 3D layer is `aria-hidden` and decorative; product information stays in real text. `prefers-reduced-motion` automatically switches off the wobble, spin, tilt and damping. Without WebGL, the stage gets `s3h-fallback` and an optional `<img class="s3h-poster">` stays visible.

VERIFY BEFORE SHIPPING

```
node .claude/skills/scroll-3d-hero/scripts/preview.mjs page.html --out /tmp/s3h --stops 0,0.25,0.5,0.75,1
node .claude/skills/scroll-3d-hero/scripts/preview.mjs http://localhost:5173/ --viewports
desktop,tablet,mobile
# --after-ready "document.querySelector('#next').click()" runs code before the screenshots
```

10 WHAT'S IN THE SKILL FOLDER

PATH	PURPOSE
`SKILL.md`	Trigger description, how it works, step-by-step workflow, option reference, pitfalls.
`assets/scroll-3d-hero.js`	The runtime: sticky-stage renderer, keyframes, anchors, idle motion, procedural plate and bowl, GLB loader, model swap, debug overlay, layout checks.
`assets/template.html`	Complete demo page (hero, statement, product card, after section) with desktop and mobile choreography.
`scripts/preview.mjs`	Playwright screenshot checker: serves local files, waits for ready and settled, exits non-zero on errors.
`references/choreography.md`	How progress maps to the page, motion principles, six recipes, and the kanaknaturals.com breakdown.
`references/models.md`	Model sources, Blender export settings, compression, look tuning, posters, custom lathe shapes.
`references/frameworks.md`	Setup for plain HTML, Vite, React/Next.js, Vue/Nuxt, Svelte, Astro and site builders.
`assets/template-animal.html`	Walking-animal demo: the Khronos sample fox walks, pauses, runs and settles in a card as you scroll.
`assets/dot-story.js`	One field of dots for the whole page: the animated character drawn as dots, an hourglass it pours through, and the wordmark it becomes.
`assets/template-dot-story.html`	The Snolep dot story page (sticky hourglass panel beside text boxes, wordmark footer).
`assets/dot-wordmark.js`	Dot-matrix footer wordmark: text or a logo drawn in dots that fly in and scatter away from the pointer. No dependencies.
`references/dot-wordmark.md`	Wordmark setup, options, logo mode, tuning, and the unitedcarriers.com breakdown.
`evals/evals.json`	Four realistic test prompts for future skill evaluations (not shipped in the .skill package).

11 DOT-MATRIX FOOTER WORDMARK

A companion effect for the bottom of the footer: the brand name drawn in small dots that fly in when the footer appears, scatter away from the cursor and spring back. It's modelled on the footer of unitedcarriers.com and lives in `assets/dot-wordmark.js`.



Fly-in. As the footer scrolls into view, the dots start from random positions and settle into the letters.



At rest. "SNOLEP TECHNOLOGIES" on one line, sized to fill the width, in a mid-grey dot colour token.



Pointer. Dots near the cursor are pushed away with a little jitter, then spring back once it stops moving.

HOW UNITEDCARRIERS.COM DOES IT

- A canvas below the legal row; their logo SVG is drawn into it and sampled every few pixels, and every opaque sample becomes a grey dot.
- The particles start at random positions and ease into place while the footer is visible.
- The mouse pushes dots out of a small radius; they return with per-particle easing and friction.
- On desktop only; phones get a static image.

WHAT THE SKILL'S VERSION ADDS

- Draws text from a font, so the name is a one-line change. Logo images also work (`image`, with `sample: 'dark'` for logos with white cut-outs).
- Stacks the words on phones and runs there too; dragging a finger scatters the dots.
- Time-based motion, identical at 60 and 120 Hz; dots stamped from a sprite; the loop stops when at rest.
- Follows the light and dark theme, and respects reduced-motion settings.



Phone. Words stack, each filling the width.



Dark theme. The dot colour is re-read when the theme changes.

USE IT

```
<canvas id="wordmark" class="dot-wordmark"
        aria-label="Snolep Technologies">
</canvas>

<script type="module">
  import { createDotWordmark }
    from './dot-wordmark.js';
  createDotWordmark('#wordmark',
    { text: 'SNOLEP TECHNOLOGIES' });
  // or { image: '/logo.svg' }
</script>

.dot-wordmark { display: block; width: 100%;
  color: var(--dots); pointer-events: none; }
```

Main options: `dotsPerCap` (finer or coarser dots), `force` and `radius` (how strongly the dots react), `stackBelow` (when to stack the words), `flyIn` and `intro` (the entrance), `fontFamily` / `fontWeight` / `fontStretch`. The preview script captures `desktop-wordmark.png` once the dots settle.

12 ANIMATED CHARACTERS

An animal or mascot can travel through the page instead of a product. The runtime plays the model's own animation clips and chooses the gait from how fast the character moves: walking when slow, running when fast, and turning to face the way it's going. Clips can also be scrubbed by the scrollbar, so a leap freezes mid-air when the visitor stops, and `still: 'pose'` keeps it motionless at rest.



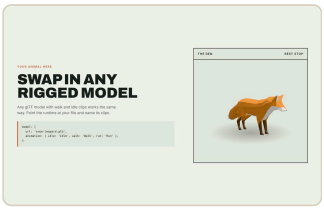
At rest. Upper right, held perfectly still (`still: 'pose'`).



Mid-leap. Arc, pitch and legs follow the scrollbar; stop here and it hangs in the air.



Landing. Lower left, nose down, shadow back under its feet.



In the den. After a run back across the page, anchored into the card.

```
model: {
  url: '/models/snow-leopard.glb',
  animation: { idle: 'Idle', walk: 'Walk',
    run: 'Run', still: 'pose' },
},
keyframes: [
  { at: 0.18, x: 0.5, y: 0.22, rz: 0 },
  { at: 0.18, clip: 'Run', clipTime: 0 },
  { at: 0.24, rz: -16 },      // nose up
  { at: 0.28, x: 0.2, y: 0.32,
    ease: 'easeOut' },        // apex
  { at: 0.36, rz: 22 },       // nose down
  { at: 0.42, x: -0.5, y: -0.62, rz: 6,
    ease: 'easeIn' },         // lands
  { at: 0.42, clip: 'Run', clipTime: 1,
    ease: 'linear' },
  { at: 0.58, x: -0.5, y: -0.62 }, // still
]
```

WHAT THE MODEL NEEDS

- A **rigged** GLB with looping idle, walk and (ideally) run clips that stay in place, facing the camera (+Z) at rest.
- Get one from a marketplace (search "snow leopard rigged animated", check the licence) or commission a 3D artist. Mixamo only rigs humanoids; AI image-to-3D gives a static mesh.
- Keep it to 1–5 MB with `gltf-transform optimize --compress meshopt --simplify false`.

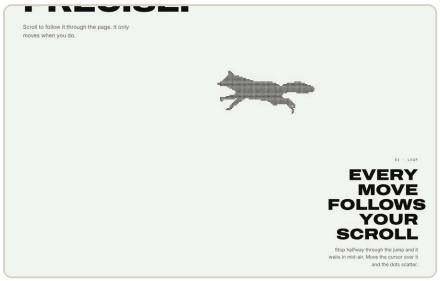
TUNING

- `walkAbove` and `runAbove` set when the gait changes; `walkSpeed` and `runSpeed` match the clips to the ground (body lengths per second).
- Use a smaller `size` on phones so crossing the screen is a few body lengths.
- Two keyframes sharing a `clip` and a `clipTime` scrub that clip with the scroll; with no jump clip, a run cycle's bound reads as a leap.
- `hero.playClip('Jump')` forces a clip; `drive: 'scroll'` walks on the spot as the page scrolls.

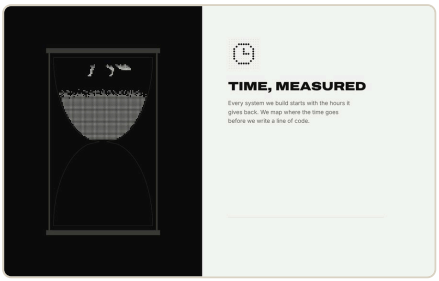
The demo fox: model by PixelMannen (CC0); rigging and animation by tomkranis (CC BY 4.0); glTF conversion by @AsoboStudio and @scurest (CC BY 4.0), from the Khronos glTF Sample Assets.

13 DOT STORY

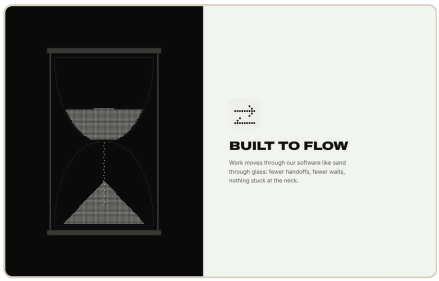
All three effects in one field of dots. The fox is rendered off screen every frame and sampled as dots, so it keeps its real gait, scrubbed leap and stillness. The same dots pour into an hourglass beside the text boxes, drain through its neck, re-form the fox at the base, and finally become the SNOLEP TECHNOLOGIES wordmark. The cursor scatters them in every form.



Leap. The dotted fox mid-bound, scrubbed by the scroll.



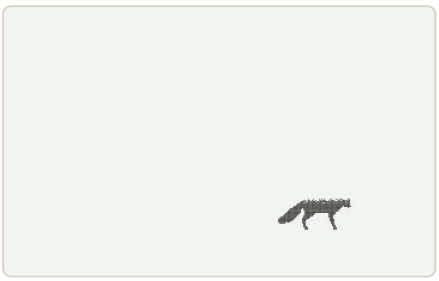
Pour. Its dots stream into the top bulb.



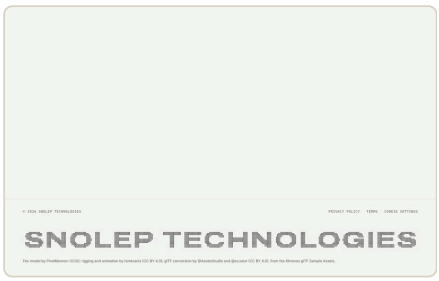
Drain. Sand falls through the neck as the boxes scroll.



Emerge. The mound shrinks from the top as the fox re-forms below.



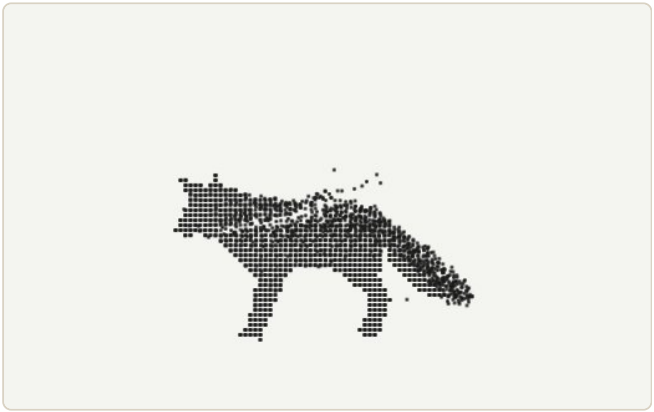
Run. It runs out, then streams down towards the footer.



Wordmark. Every dot finds its place in the name.

HOW IT WORKS

- One pool of dots, sized to the wordmark, so the last form is exact.
- Between keyframes with different `form`s each dot switches at its own moment, so shapes stream into one another.
- As `sand` goes 0 → 1, every dot steps one cell along the drain order, so the top pile sinks while a thin stream falls through the neck, all tied to scroll.
- Dots use the canvas colour, switching to `--panel-dots` inside the dark panel. The loop stops when nothing moves.



Cursor. Moving over the fox scatters its dots; they spring back into shape.

```
createDotStory({
  canvas: '#dots',
  character: { url: 'fox.glb', animation: {
    idle: 'Survey', walk: 'Walk', run: 'Run',
    still: 'pose' } },
  hourglass: { anchor: '#hourglass',
               panel: '#panel' },
  logo: { anchor: '#logo',
          text: 'SNOLEP TECHNOLOGIES' },
  keyframes: [ /* form, sand, x/y/anchor,
                  size, pitch, clip/clipTime */
],
});
```